

Projet de synthèse – Gestion de formes géométriques en 2D

Application distribuée Client C++ / Serveur Java

Université de Lorraine – Licence Informatique

Année universitaire 2025-2026

Étudiants :

- Sewa Samuel
- ZAHR Rayan

1. Introduction

Dans le cadre de l'unité d'enseignement Premiers Pas vers l'Ingénierie du Logiciel (PPIL), nous avons réalisé un projet consistant à développer une application distribuée permettant de manipuler des formes géométriques en deux dimensions.

L'objectif du projet est de mettre en pratique plusieurs concepts du génie logiciel : la programmation orientée objet, l'utilisation de design patterns et la communication réseau entre applications écrites dans des langages différents.

L'application repose sur une architecture client-serveur : le client est développé en C++ et le serveur en Java. Le client crée et manipule les formes géométriques tandis que le serveur se charge de leur affichage graphique.

2. Contexte et objectifs du projet

Le projet consiste à concevoir un système permettant de créer, manipuler et afficher des formes géométriques en deux dimensions.

Les formes simples gérées par l'application sont :

- Segment
- Cercle
- Triangle
- Polygone

L'application permet également de créer des formes composées appelées groupes, qui contiennent plusieurs formes.

Les fonctionnalités principales sont :

- création de formes

- transformations géométriques
- calcul d'aire
- dessin via un serveur Java
- sauvegarde et chargement des formes.

3. Architecture générale de l'application

L'application repose sur une architecture client-serveur.

Le client C++ est responsable de :

- la création des formes
- les transformations géométriques
- le calcul de l'aire
- l'envoi des commandes au serveur

Le serveur Java est responsable de :

- la réception des commandes
- leur interprétation
- l'affichage graphique des formes dans une fenêtre Java.

4. Architecture client-serveur

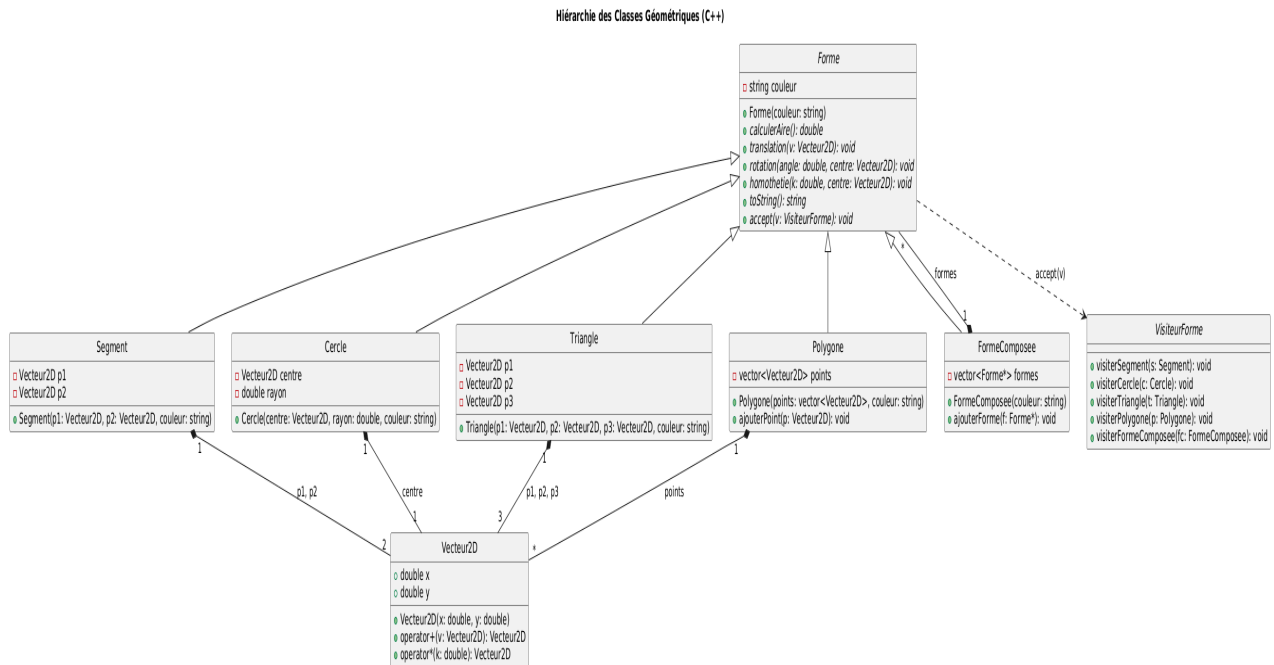
Le dessin des formes repose sur une architecture **client-serveur**.

Le client, écrit en C++, est responsable de la création des formes géométriques et de l'envoi des requêtes de dessin. Lorsqu'une forme doit être affichée, le client construit une commande sous forme de chaîne de caractères décrivant la forme et ses paramètres (par exemple un segment avec ses coordonnées et sa couleur). Cette commande est ensuite envoyée au serveur à travers une connexion TCP/IP.

Le serveur, écrit en Java, reçoit cette commande et l'analyse afin de déterminer quelle forme doit être dessinée. Pour cela, il utilise le design pattern **Chain of Responsibility**. La commande est transmise successivement à plusieurs objets appelés *handlers*, chacun étant responsable d'un type de forme particulier (segment, cercle, triangle, polygone). Lorsqu'un handler reconnaît la commande, il extrait les paramètres nécessaires et déclenche le dessin de la forme dans une fenêtre graphique Java.

Cette organisation permet de séparer clairement les responsabilités : le client gère la logique métier et la construction des formes, tandis que le serveur se charge uniquement de l'affichage graphique.

5. Hiérarchie des classes



6. Communication client-serveur

La communication entre le client C++ et le serveur Java repose sur le protocole TCP/IP. L'idée est simple : le client construit une forme en mémoire, puis lorsqu'il veut la dessiner, il ouvre une connexion réseau vers le serveur Java, lui envoie une commande sous forme de chaîne de caractères, puis le serveur interprète cette commande et effectue le dessin dans une fenêtre graphique. Ce principe correspond au sujet du projet, où la partie graphique est déléguée au serveur Java tandis que la logique métier reste en C++.

Le choix d'une chaîne de caractères comme format d'échange est pratique, car elle est facile à construire côté C++ et facile à analyser côté Java. Par exemple, la commande :

SEGMENT 10 10 200 200 2

décrit complètement une forme. Le premier mot, SEGMENT, indique le type de forme. Les nombres suivants représentent ses paramètres : ici les coordonnées des deux extrémités du segment, puis le code de la

couleur. On peut donc lire cette commande comme : « dessiner un segment allant du point (10,10) au point (200,200) avec la couleur 2 ».

Côté client, cette chaîne est construite à partir des attributs de l'objet. Une fois prête, elle est envoyée sur le socket TCP après la connexion au serveur. Ce fonctionnement correspond au modèle classique d'un client TCP/IP en C++ : initialisation, création du socket, connexion, envoi du message, puis fermeture.

Côté serveur, la chaîne reçue est lue puis découpée en plusieurs morceaux, généralement en séparant sur les espaces. Avec l'exemple précédent, on obtient les éléments suivants :

```
["SEGMENT", "10", "10", "200", "200", "2"]
```

Le serveur regarde alors le premier mot pour savoir quel traitement appliquer. Comme le projet utilise le design pattern Chain of Responsibility, la commande passe dans une chaîne de handlers. Chaque handler vérifie s'il reconnaît le type de commande. Si c'est le cas, il extrait les paramètres, les convertit en nombres, puis appelle les méthodes graphiques Java pour dessiner la forme. Sinon, il transmet la commande au handler suivant.

Cette solution a plusieurs avantages : elle est simple à déboguer, car on peut afficher directement les commandes envoyées, elle sépare clairement le client métier du serveur graphique, et elle permet d'ajouter facilement de nouvelles formes en définissant un nouveau format de commande et un nouveau handler correspondant.

7. Design patterns utilisés

Plusieurs design patterns ont été utilisés.

Visitor :

On pourrait à l'avenir souhaiter sauvegarder nos formes sous un autre format que celui implémenté à présent (format fichier.txt). C'est pour cela qu'on a choisi d'utiliser le design pattern Visitor, qui nous permet

d'implémenter une interface pour chaque format souhaité. Sans avoir à ajouter une méthode dans les classes métier pour chaque format.

Chain of Responsibility :

Le design pattern Chain of Responsibility est utilisé côté serveur Java pour traiter les commandes envoyées par le client C++. Lorsqu'une chaîne de caractères décrivant une forme est reçue, elle est transmise à une chaîne d'objets appelés handlers. Chaque handler vérifie s'il reconnaît la commande (par exemple SEGMENT, CERCLE, TRIANGLE, etc.). S'il peut la traiter, il exécute l'action correspondante (dessin de la forme). Sinon, il transmet la commande au handler suivant dans la chaîne. Cette organisation évite d'utiliser de grandes structures if ou switch et facilite l'ajout de nouveaux types de commandes.

Ce Design pattern a aussi été utile aux transformations géométriques sur les différentes formes (Segment, Cercle, Triangle et Polygone) et leur chargements à partir d'un fichier.

Singleton :

Permet de garantir qu'une ressource globale, comme l'initialisation réseau, ne soit créée qu'une seule fois dans l'application.

8. Difficultés rencontrées

Plusieurs difficultés ont été rencontrées pendant le développement.

La gestion de l'héritage en C++ a généré certaines erreurs de compilation, notamment avec les classes abstraites et les méthodes virtuelles beaucoup d'erreurs de compatibilité entre les .h et les .cpp .

Nous avons également rencontré des problèmes d'inclusions circulaires entre certains fichiers .h. (boucles d'inclusions)

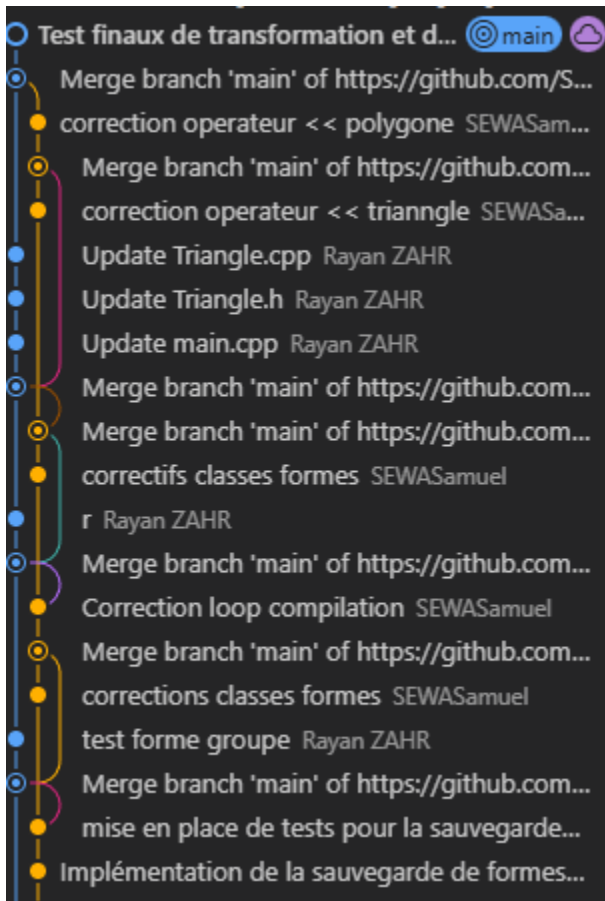
La compréhension du design pattern Visitor a également été une difficulté.

Enfin, la communication entre le client C++ et le serveur Java via TCP/IP a nécessité une bonne compréhension du fonctionnement des sockets.

9. Organisation du travail

Le projet a été réalisé en binôme.

Nous avons partagé le travail équitablement et nous avons utilisé GitHub afin de travailler efficacement et de manière cohérente .



Rayan ZAHR :

- créer les classes Segment, Cercle, Triangle et Polygone Forme etc
- implémentation du Client Serveur
- Dessin en Java

- imagination de la conception du projet
- corriger les erreurs de compilation

Samuel SEWA :

- les transformations géométriques
- la sauvegarde et le chargement
- certaines parties du serveur Java.
- imagination de la conception du projet
- corriger les erreurs de compilation

10. Conclusion

La réalisation de ce projet de gestion de formes géométriques distribuées a constitué une étape charnière dans notre formation en informatique. Elle nous a permis de confronter les concepts théoriques de la **Programmation Orientée Objet (POO)** aux réalités de l'ingénierie logicielle.

Synthèse technique

L'utilisation rigoureuse de langages aux paradigmes complémentaires — la performance et la gestion fine de la mémoire en **C++**, couplées à la richesse des bibliothèques graphiques de **Java** — a démontré l'efficacité d'une architecture client-serveur bien conçue. L'implémentation des **Design Patterns** (Visitor, Chain of Responsibility, Singleton) a garanti que notre application soit évolutive : l'ajout d'une nouvelle forme ou d'un nouveau type de rendu ne nécessite pas de restructuration lourde du code existant. Quelqu'un peut passer après nous et rajouter des fonctionnalités sans devoir repasser derrière nous.

Apports personnels et collaboration

Au-delà de l'aspect purement technique, ce travail en binôme a été une expérience enrichissante. La gestion des conflits via **Git** et la nécessité de définir un protocole de communication réseau strict nous ont sensibilisés à l'importance de la documentation et de la rigueur contractuelle entre les différentes briques d'un logiciel. Les difficultés

rencontrées, notamment sur la gestion des dépendances en C++ et la synchronisation des flux TCP, ont été des vecteurs d'apprentissage essentiels pour notre future carrière.

Perspectives d'évolution

Bien que l'application soit pleinement fonctionnelle, plusieurs pistes d'amélioration pourraient être explorées :

- **Performance** : Passage en UDP pour le temps réel : Pour des manipulations très fluides, le protocole UDP pourrait être envisagé pour sa rapidité, quitte à perdre quelques paquets de rafraîchissement.
- **Fonctionnalités** : L'ajout d'une interface graphique côté client (via Qt par exemple) pour permettre une manipulation interactive des formes à la souris.

En définitive, ce projet confirme que la robustesse d'un logiciel ne repose pas uniquement sur la qualité de son code, mais sur la clarté de sa conception et de son architecture.